NI1 – Programação Orientada a Objetos: Documentação do Jogo Space Invaders

Turma: 2NAADS
Data da apresentação: 23/10/2026
Integrantes: [nome 1], [nome 2], [nome 3], [nome 4]
Tecnologia: JavaScript com `<canvas>` (a versão atual é um projeto em vários arquivos, uma classe por arquivo; as primeiras versões eram arquivos únicos)

Requisitos do professor e onde foram atendidos

Atualizado em 12/10/2026 com o Guia do grupo, o repositório do GitHub e as versões mais recentes do jogo (hordas, som, música e tela cheia). O trabalho cobre os requisitos do enunciado da NI1; faltam os prints de antes e depois de cada integrante, o prompt do Gemini e os nomes do grupo.

Requisito da NI1	Situação	Onde neste documento
Grupo de 3 a 5 integrantes	[confirmar nomes]	Capa e seção 7
Jogo em JavaScript com `<canvas>`	Atendido	Seção 1
Classes e objetos	Atendido	Seção 2.2
Herança	Atendido	Seção 2.3
Pelo menos um tipo de polimorfismo	Atendido (sobrescrita de métodos)	Seção 2.4
Código organizado, comentado e bem nomeado	Atendido	Arquivo do jogo
Duas ferramentas de IA, com comparação	Atendido (Gemini e Claude)	Seções 3 e 6
Prompts, resultados, bugs, correções e código antes e depois	Parcial: faltam o prompt do Gemini e os bugs achados nos testes	Seções 4 e 5

Requisito da NI1	Situação	Onde neste documento
Prints, vídeos ou GIFs	Parcial: há capturas do jogo (PDFs e PNGs), mas faltam os prints de antes e depois de cada integrante, nomeados como o guia pede	Seções 1 e 4
Cada integrante explica sua contribuição e domina o código	A cumprir em 23/10/2026	Seção 7
Assets e tutoriais de terceiros com créditos	Atendido: nenhum foi usado	Seção 12
Jogo funcionando no dia da apresentação	Testado de forma automática no navegador; o grupo deve jogar e testar à mão antes de 23/10/2026	Seção 1

1. Visão geral

O grupo desenvolveu um jogo no estilo Space Invaders. O jogador controla uma nave na parte de baixo da tela e deve destruir ondas de inimigos que descem em formação. O jogo termina em vitória ao chegar a 1000 pontos, ou em derrota se as vidas acabarem ou se os inimigos alcançarem a nave.

Tecla	Ação
Setas ou A/D	Mover a nave
Espaço	Atirar
P ou Esc (ou botão)	Pausar e continuar
Enter	Iniciar ou jogar de novo

[inserir print do menu] [inserir print da partida]

2. Conceitos de POO aplicados

2.1 Hierarquia de classes

```
Entidade
├── Projetil
├── Jogador
├── PowerUp
└── Inimigo
    ├── InimigoBasico
    ├── InimigoResistente
    └── InimigoAtirador
```

2.2 Classes e atributos

Classe	Herda de	Atributos principais	Função
`Entidade`	–	`x`, `y`, `largura`, `altura`, `cor`, `vivo`	Base de todos os objetos; tem `desenhar()`, `atualizar()` e `colideCom()`
`Projetil`	`Entidade`	`velocidadeY`	Tiro do jogador (sobe) e dos inimigos (desce)
`Jogador`	`Entidade`	`velocidade`, `vidas`, `recarga`, `invulneravel`	Nave controlada pelo usuário
`PowerUp`	`Entidade`	`velocidade`	Cai de inimigos destruídos e dá 1 vida
`Inimigo`	`Entidade`	`pontos`, `vida`	Comportamento padrão dos inimigos
`InimigoBasico`	`Inimigo`	–	Vermelho, 10 pontos, 1 tiro para destruir, não atira
`InimigoResistente`	`Inimigo`	–	Amarelo, 20 pontos, aguenta 2 tiros
`InimigoAtirador`	`Inimigo`	–	Roxo, 30 pontos, atira no jogador

2.3 Herança

`Jogador`, `Projetil`, `PowerUp` e `Inimigo` herdam de `Entidade` e reaproveitam posição, tamanho e colisão por meio de `super(...)`. Há dois níveis de herança: `Entidade` → `Inimigo` → três tipos de inimigo.

2.4 Polimorfismo (sobrescrita de métodos)

A classe `Inimigo` define comportamentos padrão, e cada filha os sobrescreve quando precisa:

```
class Inimigo extends Entidade {
    tentarAtirar() { return null; }   // padrão: não atira
}

class InimigoAtirador extends Inimigo {
    tentarAtirar() {                  // sobrescreve: devolve um projétil
        return new Projetil(this.x + this.largura / 2 - 2, this.y +
this.altura, 5, '#c266ff');
    }
}
```

O jogo sorteia um inimigo e chama `sorteado.tentarAtirar()` sem saber o tipo dele: o básico devolve `null` e o atirador devolve um `Projetil`. O mesmo vale para `desenhar()`, que é diferente em cada tipo de inimigo, e para `levarTiro()`, que muda no inimigo resistente.

2.5 Encapsulamento

Cada classe controla o próprio estado. Por exemplo, o jogo chama `jogador.levarDano()` em vez de alterar `vidas` e `invulneravel` por fora.

3. Ferramentas de IA utilizadas

IA	Uso no projeto
Gemini	Gerou o código-base "Sobrevivência Espacial" (desviar de blocos)
Claude	Evoluiu a base para um Space Invaders completo, com tiros, ondas e novas classes

Observação: o Space Invaders do Claude foi construído a partir do código do Gemini. Portanto, esta comparação mostra a base do Gemini contra a versão evoluída pelo Claude, e não duas IAs recebendo o mesmo pedido.

4. Diário de desenvolvimento

Etapa 1 – Código-base (Gemini)

- **Prompt usado:** [colar aqui o prompt exato enviado ao Gemini]
- **Resultado:** jogo "Sobrevivência Espacial": a nave desvia de blocos vermelhos e apanha blocos verdes, com vitória aos 500 pontos.
- **Classes geradas:** `Entidade`, `Jogador`, `Inimigo`, `PowerUp`.
- **Evidência:** [inserir print]

Etapa 2 – Criação do Space Invaders (Claude)

- **Prompt usado:** "crie um jogo space-invaders a partir deste código" (com o código do Gemini anexado)
- **Resultado:** jogo com formação de inimigos, tiros, vidas, ondas, power-up e 3 tipos de inimigo.
- **Evidência:** [inserir print e GIF da partida]

Etapa 3 – Botão de pausa (Claude)

- **Prompt usado:** "coloque um botão de pausa no jogo"
- **Resultado:** botão "Pausar/Continuar", teclas P e Esc, tela "PAUSADO" e pausa automática ao trocar de aba.
- **Como funciona:** `alternarPausa()` só muda o estado entre `'jogando'` e `'pausado'`; o loop principal só atualiza a lógica no estado `'jogando'`.
- **Evidência:** [inserir print]

Etapa 4 – Primeira versão do Space Invaders (Claude)

- **Prompt usado:** "um jogo space invader"
- **Resultado:** `space-invaders.html`, em arquivo único, com a classe `Jogo`, as classes `Entidade`, `Jogador`, `Projetil` e inimigos comuns e atiradores, formação em ondas e desenhos em pixels. Foi feita antes da Etapa 2 e não usou o código do Gemini.
- **Evidência:** `capturas-jogo-claude-original.pdf` (menu, partida e fim de jogo; o Game Over aconteceu de forma natural).

Etapa 5 – Comparação entre Gemini e Claude, e capturas

- **Prompt usado:** "importe o código do jogo que fiz no gemini e compare com o seu"
- **Resultado:** a comparação da seção 6 e o PDF de capturas do jogo do Gemini, que confirmou em teste os bugs 1 e 2.
- **Evidência:** `capturas-jogo-gemini.pdf` e `capturas-space-invaders.pdf`.

Etapa 6 – Pausa e visual clássico no projeto em vários arquivos (zip "escudos e velocidade")

- **Prompt usado:** "Adiciona nesse codigo apenas a funcionalidade de botão de pause, e adiciona um visual parecido com o space invaders original, contendo os aliens e nave"
- **Resultado:** pausa em `Jogo.js` e `Tela.js` (estado `pausado` e métodos `pausar()`, `continuar()` e `alternarPausa()`); novo arquivo `Sprites.js` com a lula, o caranguejo e o polvo (2 quadros cada, para "andar") e o canhão da nave. `Inimigo.desenhar()` e `Jogador.desenhar()` sobrescrevem o desenho da `Entidade`.
- **Testes:** no navegador, a pausa congela o jogo, Enter durante a pausa não reinicia a partida e a pausa automática ao trocar de aba funciona.
- **Evidência:** `pausa_antes.png`, `pausa_depois.png`, `visual_inicio.png` e `visual_partida.png`.

Etapa 7 – Som e 5 níveis de dificuldade (projeto com a pausa)

- **Prompt usado:** "Coloque som no jogo e dificulte o nível do jogo até o quinto nivel"
- **Resultado:** `Som.js`, com efeitos sintetizados pela Web Audio API (tiro, tiro inimigo, explosão, nave atingida, novo nível, pausa, vitória, derrota e a "marcha" dos invasores), e `Nivel.js`, com 5 níveis de velocidade, tiros e fileiras crescentes. Tecla e botão M ligam e desligam o som.
- **Testes:** o tiro toca som; no mudo nenhum som novo é criado; os 5 níveis levam à vitória; o reinício volta ao nível 1.
- **Evidência:** `nivel_1_aviso.png`, `nivel_3_aviso.png`, `nivel_5_jogando.png` e `vitoria.png`.

Etapa 8 – O som não saía

- **Prompts usados:** "por que ão sai o audio" e, mais tarde, "não sai o som"
- **Resultado:** medição do sinal de cada efeito e correção do volume (bug 5), e depois um aviso no botão quando o navegador bloqueia o áudio (bug 6).

Etapa 9 – Projeto "ordas" (versão lateral): som e dificuldade por horda

- **Prompt usado:** "use este arquivo para colocar som e dificultar os níveis do jogo" (com o zip `space-invaders-ordas.zip`)
- **Resultado:** `Som.js` adaptado e uma tabela `Jogo.HORDAS` com a dificuldade de cada horda (velocidade, avanço, chance e velocidade dos tiros). Medi quanto tempo cada horda leva para os invasores passarem e testei com um robô simples (seção 8.3).
- **Evidência:** `som_aviso_horda1.png`, `som_aviso_horda4.png`, `som_horda5.png` e `som_vitoria.png`.

Etapa 10 – Tela cheia

- **Prompt usado:** "estenda a tela"
- **Resultado:** botão e tecla F para tela cheia; em tela cheia o título e as instruções saem para o jogo ganhar o espaço. Pode não funcionar dentro de uma pré-visualização em moldura; nesse caso vale o F11 do navegador.
- **Evidência:** `tela_normal.png` e `tela_cheia.png`.

Etapa 11 – Música e hordas mais lentas

- **Prompt usado:** "quero ou música e as fases níveis de dificuldade está muito rapida"
- **Resultado:** `Musica.js` (música de fundo sintetizada, tecla e botão N) e a dificuldade das hordas ficou mais lenta e mais gradual (seção 8.3).
- **Evidência:** [gravar um vídeo curto com som, porque a música não aparece em print]

Etapa 12 – Horda 5 mais lenta (retorno de um colega)

- **Prompt usado:** "meu colega disse que o nível 5 está muito rápido, diminua a velocidade do jogo nesse nível, diminua o ataque dos inimigos."
- **Resultado:** a velocidade, o avanço, a chance de tiro e a velocidade dos tiros da horda 5 foram reduzidos. Uma primeira tentativa, com velocidade 0,65, fez o robô de teste perder; a escolha final foi validada em 10 partidas do robô, que venceu todas.
- **Código antes e depois:**

```
// Antes
{ colunas: 'AARAABAA', porColuna: 10, velocidade: 0.8, avanco: 44,
chanceTiro: 0.5, velocidadeTiro: 1.05 },
// Depois
{ colunas: 'AARAABAA', porColuna: 10, velocidade: 0.7, avanco: 38,
chanceTiro: 0.35, velocidadeTiro: 0.9 },
```

5. Bugs encontrados e correções

Bug 1 – CSS sem seletor (código do Gemini)

- **Descrição:** as regras `margin`, `padding` e `box-sizing` estavam soltas dentro do `<style>`, sem seletor, e por isso eram ignoradas pelo navegador. Efeito medido no teste: a regra do body que vem logo depois (fundo escuro e fonte Courier) também deixou de valer, e a página aparece com fundo branco e fonte com serifa.
- **Antes:**

```
margin: 0;
padding: 0;
box-sizing: border-box;
```

- **Depois:**

```
* { margin: 0; padding: 0; box-sizing: border-box; }
```

Bug 2 – Tela de "Game Over" não some ao reiniciar (código do Gemini)

- **Descrição:** em `reiniciarJogo()`, a classe `escondido` era adicionada e logo em seguida `overlay.className = ''` a apagava. O jogo reiniciava por baixo, mas a tela de fim continuava visível. Confirmado em teste: depois de clicar em Jogar Novamente, o overlay seguiu com display flex e o jogo rodando por baixo.
- **Antes:**

```
overlay.classList.add('escondido');
overlay.className = '';
```

- **Depois:**

```
overlay.className = 'escondido';
```

Bug 3 – Fim de jogo podia ser chamado duas vezes (código do Gemini)

- **Descrição:** o loop não parava depois de `fimDeJogo(false)`. Se no mesmo quadro a pontuação chegasse a 500, `fimDeJogo(true)` também era chamado e mostrava "VITÓRIA" depois de uma derrota.
- **Correção na versão do Claude:** o fim de jogo é decidido com `if / else if`, então só um dos casos acontece por quadro.

Bug 4 – CSS sem uso (código do Gemini)

- **Descrição:** as regras `.inimigo.basico`, `.inimigo.rapido` e `.inimigo.atirador` não eram aplicadas a nenhum elemento.
- **Correção:** os tipos de inimigo passaram a existir de verdade como classes (`InimigoBasico`, `InimigoResistente`, `InimigoAtirador`) e as regras sem uso foram removidas.

Bugs encontrados pelo grupo nos testes

Os bugs 5 e 6 abaixo apareceram durante o desenvolvimento das versões com som. O grupo deve acrescentar aqui os que encontrar ao jogar, com descrição, código antes e depois e print.

Bug 5 – Som baixo demais (versão com som)

- **Sintoma:** depois de adicionado o som, o jogador relatou que não ouvia nada.
- **Causa (medida no navegador):** o áudio estava ativo e os sons eram disparados, mas o volume geral era 0,18 e cada efeito chegava a um pico entre 0,04 e 0,14 (de 1,0). A marcha dos invasores usava notas de 82 a 110 Hz, que alto-falantes pequenos quase não reproduzem.
- **Correção:** volume geral 0,6, compressor para evitar estalos, marcha em onda quadrada (98 a 130 Hz, com harmônicos mais agudos), efeitos mais altos e um bipe de teste ao ligar o som.
- **Pico de sinal, antes e depois:** tiro do jogador 0,087 → 0,289; tiro inimigo 0,041 → 0,274; explosão 0,112 → 0,384; nave atingida 0,143 → 0,509; marcha 0,136 → 0,286; novo nível 0,069 → 0,402.

Antes:

```
this.volume = 0.18;
// marcha
this._tom('triangle', nota, nota, 0.09, 0.9);
```

Depois:

```
this.volume = 0.6;
// marcha (onda quadrada)
this._tom('square', nota, nota, 0.09, 0.5);
```

Bug 6 – Áudio possivelmente bloqueado pelo navegador

- **Sintoma:** o jogador relatou de novo que o som não saía.

- **Causa provável:** os navegadores só liberam o áudio depois de um clique ou tecla do jogador, e o Safari do iPhone ainda exige que um som de silêncio toque dentro do toque. Isso não foi confirmado no aparelho do jogador. [preencher: o botão mostrou "Som: ligado" ou "Som bloqueado"?]
- **Correção:** o áudio é liberado no primeiro clique, toque ou tecla em qualquer lugar da página, e o botão passa a mostrar "Som bloqueado: clique aqui" quando o navegador impede. Testado simulando um navegador que bloqueia o áudio.

Antes:

```
if (this.contexto.state === 'suspended') this.contexto.resume();
```

Depois:

```
if (this.contexto.state === 'suspended') {
  return this.contexto.resume().then(() => this.contexto.state).catch(() =>
'suspended');
}
return Promise.resolve(this.contexto.state);
```

6. Comparação entre as IAs

Critério	Gemini (Sobrevivência Espacial)	Claude (Space Invaders)
Tipo de jogo	Desviar de blocos que caem; sem tiro	Space Invaders: formação, tiros, ondas, vidas
Classes	4	8
Herança	Um nível	Dois níveis
Polimorfismo	Fraco: `atualizar()` repetido e `desenhar()` nunca sobrescrito	Mais claro: `tentarAtirar()`, `levarTiro()` e `desenhar()` sobrescritos
Comentários	Poucos no JavaScript	Comentários nas classes e nos blocos principais
Velocidade	Atrelada aos quadros por segundo	Escala de tempo; mesma velocidade em qualquer monitor
Layout	Largura fixa	Se adapta à tela, com botões de toque
Simplicidade	Mais curto e fácil de explicar	Mais longo, exige mais estudo do grupo

Conclusão

O código do Gemini foi uma base simples e organizada, boa para começar, mas cobria o mínimo de POO e continha bugs. A versão do Claude ampliou a hierarquia e deixou o polimorfismo mais evidente. Como contrapartida, é maior e mais complexa, e a lógica do jogo está em funções globais, e não em uma classe `Jogo`. [o grupo pode acrescentar aqui a própria avaliação após testar os dois]

Limitação: a análise dos códigos foi feita pela leitura do programa. O grupo deve testar os dois jogos e registrar os resultados.

7. Contribuição individual

Integrante	O que fez	Classes ou partes que domina
[nome 1]	[descrever]	[descrever]
[nome 2]	[descrever]	[descrever]

Integrante	O que fez	Classes ou partes que domina
[nome 3]	[descrever]	[descrever]
[nome 4]	[descrever]	[descrever]

Registro de contribuição: [nome]

Este registro segue o modelo do Guia do grupo (NI1 – Space Invaders), para entrar na documentação final sem retrabalho. O que ainda não se aplica ou está pendente aparece entre colchetes.

1. O que escolhi fazer

- Tipo: melhoria
- Descrição: pedi à IA e integrei o botão de pausa (botão, teclas P e Esc, tela "PAUSADO" e pausa automática ao trocar de aba) e, depois, o som, a música de fundo, a dificuldade crescente por horda (inclusive o ajuste da horda 5 pedido por um colega) e a tela cheia. A pontuação e a tela de vitória já vinham do código-base; eu integrei e testei.
- Arquivos: space-invaders-ni1.html (versão em arquivo único) e, no projeto em vários arquivos, Jogo.js, Tela.js, Inimigo.js, InimigoAtirador.js, index.html e style.css, mais os novos Som.js e Musica.js

2. Uso de IA

- Ferramenta usada: Claude
- Prompts enviados: "coloque um botão de pausa no jogo" e os demais, na íntegra, na seção 10
- Resposta resumida da IA: adicionou o botão Pausar/Continuar, as teclas P e Esc, a tela "PAUSADO" e a pausa automática ao trocar de aba, e fez o Enter só iniciar ou reiniciar fora da partida.
- Segunda ferramenta: Gemini (gerou o código-base "Sobrevivência Espacial"; [colar aqui o prompt usado no Gemini])
- Comparação entre as duas: está na seção 6 do documento do grupo (Gemini x Claude).

3. Bug encontrado

- Sintoma: depois de adicionado o som, o jogador relatou "não sai o som" (seção 5, bugs 5 e 6)
- Causa: volume geral baixo (0,18) e marcha dos invasores em notas graves demais para alto-falantes pequenos; um possível bloqueio do áudio pelo navegador ainda não foi confirmado

- Como corrigi: pedi a correção à IA: volume 0,6, compressor, marcha em onda quadrada e um botão que avisa quando o navegador bloqueia o áudio

4. Código antes e depois

Antes (tecla Enter, e a pausa ainda não existia):

```
if (e.code === 'Enter' && estado !== 'jogando') iniciarJogo();
```

Depois (Enter só fora da partida, atalhos P e Esc, e a nova função):

```
if (e.code === 'Enter' && (estado === 'inicio' || estado === 'fim'))
iniciarJogo();
if ((e.code === 'KeyP' || e.code === 'Escape') && !e.repeat) alternarPausa();

// Pausa: alterna o estado entre 'jogando' e 'pausado'
function alternarPausa() {
    if (estado === 'jogando') estado = 'pausado';
    else if (estado === 'pausado') estado = 'jogando';
    else return;
    botaoPausa.innerText = estado === 'pausado' ? '▶ Continuar' : '⏸ Pausar';
    teclas['Space'] = false;
}
```

5. Evidências

- Print, vídeo ou GIF antes da alteração: pausa_antes.png (projeto sem a pausa); renomear com o seu nome, como o guia pede
- Print, vídeo ou GIF depois da alteração: pausa_depois.png, visual_partida.png, som_horda5.png e tela_cheia.png (nas pastas prints dos zips entregues); renomear com o seu nome

6. O que entendi e vou explicar na apresentação

- Como a minha parte funciona: o jogo guarda um estado ('inicio', 'jogando', 'pausado' ou 'fim'). A função alternarPausa() troca entre 'jogando' e 'pausado'. O loop principal roda sempre, mas só chama atualizarJogo() quando o estado é 'jogando'; em 'pausado' ele só desenha, e por isso tudo congela na tela. O HUD mostra pontos, vidas e onda, e fimDeJogo(true) exibe a vitória ao chegar a 1000 pontos.
- Conceito de POO que aparece: a pausa em si controla o estado do jogo em funções, e não em uma classe. Mesmo pausado, o jogo mostra o polimorfismo: desenharJogo() chama desenhar() em cada objeto (jogador, inimigos, tiros) sem saber o tipo, e cada classe se desenha do seu jeito.

7. Créditos

- Nenhum asset, tutorial ou código de terceiros além do código-base gerado pelo Gemini e evoluído com o Claude (seções 3 e 4).

Perguntas que o professor pode fazer

Pergunta	Resposta curta
Onde está a herança?	`Jogador`, `Projetil`, `PowerUp` e `Inimigo` herdam de `Entidade` e reaproveitam posição, tamanho e colisão com `super(...)`. Os três tipos de inimigo herdam de `Inimigo`.
Onde está o polimorfismo?	`tentarAtirar()` devolve `null` em `Inimigo` e um `Projetil` em `InimigoAtirador`; o jogo chama o método sem saber o tipo do inimigo. `desenhar()` e `levarTiro()` também mudam por tipo.
O que é encapsulamento no jogo?	O jogo chama `jogador.levarDano()` em vez de mexer em `vidas` e `invulneravel` por fora.
Como funciona a pausa?	`alternarPausa()` troca o estado entre `'jogando'` e `'pausado'`, e o loop só atualiza a lógica quando o estado é `'jogando'`.
Por que o loop usa uma escala de tempo?	Para o jogo ter a mesma velocidade em monitores de 60 Hz, 144 Hz ou outros.
Qual bug vocês corrigiram?	O reinício deixava a tela de "Game Over" visível porque `classNome = ''` apagava a classe `escondido` recém-adicionada (seção 5, bug 2).

8. Versão atual do jogo (projeto lateral, com hordas, som e música)

A versão atual é o projeto em vários arquivos, com uma classe por arquivo. Os invasores vêm da direita em hordas, a nave anda nos 4 sentidos e o jogo termina em vitória ao sobreviver às 5 hordas.

8.1 Controles

Tecla	Ação
Setas ou W A S D	Mover a nave nos 4 sentidos

Tecla	Ação
Espaço	Atirar
P ou Esc (ou botão)	Pausar e continuar
M (ou botão)	Ligar e desligar todo o som, música incluída
N (ou botão)	Ligar e desligar só a música
F (ou botão)	Tela cheia
Enter	Iniciar ou jogar de novo

8.2 Arquivos e classes

Arquivo	Classe	Herda de	Função
`Entidade.js`	`Entidade`	–	Base: posição, tamanho, `desenhar()`, `atualizar()` e `colideCom()`
`Projetil.js`	`Projetil`	`Entidade`	Tiros do jogador e dos inimigos
`Jogador.js`	`Jogador`	`Entidade`	Nave: movimento, tiro e vidas
`Inimigo.js`	`Inimigo`	`Entidade`	Base dos invasores: pontos, desenho por sprite e multiplicadores de tiro por horda
`InimigoBasico.js`	`InimigoBasico`	`Inimigo`	10 pontos, não atira
`InimigoRapido.js`	`InimigoRapido`	`Inimigo`	20 pontos, mais rápido
`InimigoAtirador.js`	`InimigoAtirador`	`Inimigo`	30 pontos, sobrescreve `tentarAtirar()`
`Barreira.js`	`Barreira`	`Entidade`	Peças dos escudos, com resistência 3
`Tela.js`	`Tela`	–	Telas por cima do canvas (início, fim, pausa e aviso de horda)
`Jogo.js`	`Jogo`	–	Orquestra tudo: hordas, colisões, pontos, pausa, som e tela cheia

Arquivo	Classe	Herda de	Função
`Sprites.js`	`Sprites`	–	Desenhos em pixel art (já vinham no projeto recebido)
`Som.js`	`Som`	–	Efeitos sonoros sintetizados (novo)
`Musica.js`	`Musica`	–	Música de fundo sintetizada (novo)

Polimorfismo: o `Jogo` chama `mover()`, `tentarAtirar()` e `desenhar()` em todos os invasores sem saber o tipo; o básico não atira e o atirador devolve um `Projetil`.

8.3 Hordas e dificuldade

Cada horda tem a sua dificuldade na tabela `Jogo.HORDAS`: velocidade do bloco, avanço do bloco a cada batida no topo ou no chão, multiplicador da chance de tiro e multiplicador da velocidade dos tiros. O "tempo parado" é quanto os invasores levam para passar com o jogador sem agir e sem escudos, medido no navegador.

Horda	Invasores	Atiradores	Velocidade	Avanço	Chance de tiro	Velocidade do tiro	Tiros por segundo (estimado)	Tempo parado	Tempo parado no projeto original
1	32	16	0,85	44	0,8	0,85	1,5	53 s	38 s
2	40	24	0,9	46	0,8	0,9	2,3	45 s	36 s
3	48	32	0,95	48	0,75	0,95	2,9	39 s	35 s
4	56	40	1,0	50	0,7	1,0	3,4	33 s	32 s
5	80	60	0,7	38	0,35	0,9	2,5	37 s	19 s

Histórico: uma primeira versão mais difícil deixou as hordas bem mais rápidas (35, 28, 23, 21 e 15 s) e foi considerada rápida demais; em seguida, a horda 5 ainda foi suavizada a pedido de um colega (28 s → 37 s; tiros por segundo de 3,6 → 2,5).

Teste com um robô simples (mira no invasor mais próximo, atira sem parar e desvia dos tiros): ele passa as hordas 1 a 4 sem perder vida e venceu 10 de 10 partidas na horda 5, terminando com 2,6 vidas em média. O robô não joga como uma pessoa, e pequenas mudanças na horda 5 deram resultados irregulares para ele, então o equilíbrio deve ser confirmado jogando.

8.4 Som e música

- `Som.js`: todos os efeitos são gerados na hora pela Web Audio API (osciladores e ruído), sem arquivos de áudio. Há efeitos para tiro do jogador, tiro inimigo, explosão, nave atingida, início de horda, pausa, vitória e derrota, mais a "marcha" dos invasores, que acelera quando restam poucos. O som de tiro inimigo é limitado a cerca de 7 por segundo para não virar um chiado.
- `Musica.js`: loop de 4 compassos em lá menor (Lá menor, Fá, Dó e Sol), com baixo e arpejo. O andamento começa em 100 batidas por minuto e sobe 5 a cada horda. A música para na pausa e no fim da partida e recomeça a cada partida nova.
- Os navegadores só liberam o áudio depois de um clique ou tecla; o botão de som mostra "Som bloqueado: clique aqui" quando isso não acontece (bug 6).

9. Repositório do grupo e contribuições dos colegas

O repositório do grupo no GitHub é `2026-2-NADS2/Projeto13`, na pasta `POO N1`, na branch `main`. Os colegas enviam os arquivos por Add file > Upload files, com uma pasta por contribuição. Segundo a captura de tela, o conteúdo da pasta é:

Item	O que é
`escudos e velocidade`	Pasta com os escudos e a velocidade fixa (detalhada abaixo)
`space invader (tela star e game over)`	Pasta com a tela inicial e a de Game Over (conteúdo não analisado neste documento)
Comparação Space Invaders do Claude e do Ge...	Documento de comparação entre as duas IAs
Guia do grupo como registrar sua contribuição	O guia do grupo (PDF)
`space_invaders(CLAUDE).zip` e `space_invaders(GEMINI).zip`	O jogo de cada IA, compactado

Contribuição "escudos e velocidade" (zip analisado)

- **Escudos:** nova classe `Barreira` (herda de `Entidade`), peças de resistência 3 que formam 4 escudos no formato clássico de 10 colunas. Os tiros do jogador e dos inimigos perdem força ao acertar o escudo, e os invasores que descem "atropelam" as peças.

- **Velocidade fixa:** o `Jogo` passou a usar um passo fixo de 60 atualizações por segundo (`tempoAcumulado`, `ultimoTempo`, `_atualizar()` e `_loop()`), para o jogo ter a mesma velocidade em monitores de 60, 120 ou 144 Hz.
- **Arquivos alterados:** `Jogo.js` e `index.html` (tag do `Barreira.js`); o zip traz `Jogo_antes.js`, `index_antes.html` e os prints `marcos_antes.png` e `marcos_depois.png`.
- **Autoria:** o último envio visível na captura foi feito por `Marquinhos250`, com a mensagem "Add files via upload", 3 dias antes da captura. [confirmar com o grupo quem fez cada pasta]

10. Prompts utilizados (inclusive os que não deram certo)

O guia pede o prompt exatamente como foi enviado, inclusive os que falharam. Os prompts abaixo foram enviados ao Claude.

Nº	Prompt (como enviado)	Resultado	Deu certo?
0	[colar o prompt enviado ao Gemini]	Jogo "Sobrevivência Espacial" (Etapa 1)	Sim, com os bugs 1 a 4
1	"um jogo space invader"	Primeira versão do Space Invaders (Etapa 4)	Sim
2	"crie um jogo space-invaders a partir deste código"	Space Invaders sobre o código do Gemini (Etapa 2)	Sim
3	"coloque um botão de pausa no jogo"	Botão de pausa (Etapa 3)	Sim
4	"importe o código do jogo que fiz no gemini e compare com o seu"	Comparação e capturas (Etapa 5)	Sim
5	"Adiciona nesse codigo apenas a funcionalidade de botão de pause, e adiciona um visual parecido com o space invaders original, contendo os aliens e nave"	Pausa e sprites no projeto em vários arquivos (Etapa 6)	Sim
6	"acrescente no código botões de letra"	Não atendido: o pedido ficou ambíguo e foi substituído pelo prompt 5	Não

Nº	Prompt (como enviado)	Resultado	Deu certo?
7	"Coloque som no jogo e dificulte o nível do jogo até o quinto nivel"	Som e 5 níveis (Etapa 7)	Sim
8	"por que ão sai o audio"	Medição do sinal e correção do volume (bug 5)	Parcial: depende de confirmar no aparelho
9	"use este arquivo para colocar som e dificultar os níveis do jogo"	Som e dificuldade por horda no projeto "ordas" (Etapa 9)	Sim
10	"estenda a tela"	Tela cheia (Etapa 10)	Sim, com a ressalva da pré-visualização
11	"não sai o som"	Aviso de áudio bloqueado (bug 6)	Parcial: depende de confirmar no aparelho
12	"quero ou música e as fases níveis de dificuldade está muito rapida"	Música e hordas mais lentas (Etapa 11)	Sim
13	"meu colega disse que o nível 5 está muito rápido, diminua a velocidade do jogo nesse nível, diminua o ataque dos inimigos."	Horda 5 mais lenta (Etapa 12)	Sim

11. Checklist final do guia do grupo

Item do guia	Situação
Escolhi uma melhoria ou bug e avisei o grupo	[confirmar]
Guardei uma cópia do arquivo original antes de alterar	Sim: as pastas `antes` dos zips têm as cópias
Tirei print ou GIF antes e depois	Parcial: faltam os prints com o seu nome (`nome_antes.png` e `nome_depois.png`)
Colei os prompts completos, inclusive os que falharam	Sim, na seção 10; falta o prompt do Gemini
Usei pelo menos uma IA e anotei qual	Sim: Claude, e o código-base do Gemini

Item do guia	Situação
O jogo abre e funciona com a minha alteração	Testado de forma automática; falta o teste à mão
Comentei o código novo e mantive a indentação	Sim
Consigo explicar com minhas palavras o que o código faz	A cumprir: ver as perguntas da seção 7
Informei assets, tutoriais ou código de terceiros	Sim, na seção 12: nenhum
Todos os integrantes presentes na apresentação	Combinar quem explica cada parte

Regra do guia: o `Jogo.js` é o arquivo que mais dá conflito e só uma pessoa por vez deve alterá-lo. Como ele foi alterado nas etapas 6 a 12, avise o grupo antes de enviar ao repositório.

12. Créditos e referências

- Código-base gerado pelo Gemini e evoluído com o Claude, conforme descrito nas seções 3 e 4.
- Não foram usados assets nem tutoriais de terceiros: os gráficos são desenhados por código.
- Som e música: gerados por código com a Web Audio API, sem arquivos de áudio. O visual dos invasores é inspirado no Space Invaders original (Taito, 1978); os desenhos são feitos por código, sem arquivos de imagem. [acrescentar qualquer outra fonte usada pelo grupo]